

given sentence into words using the NLTK library.

```
# import word_tokenize method from nltk
from nltk import word_tokenize
# Create a string input
sentence = "Python is a high-level programming language"
```

Now, using `word_tokenize()` one can split the sentence into its constituent words.

```
words = word_tokenize(sentence)
print(words)
['Python', 'is', 'a', 'high', 'level', 'programming',
'language']
```

In addition, `SyllableTokenizer()` in the NLTK library can be used to split a word into its tokens according to its syllables.

Stemming

The English language often uses inflected forms of different words as many English words are derived from other words; for example, the inflected word ‘generosity’ is derived from the word ‘generous’, which is the root form. All the inflected words consist of words with common root forms, but the degree of inflexion varies based on the language. For example, there are four inflected words for the following sentence — overwhelmed, generosity, friends, neighbours — all of which can be identified and transformed into their root form with the NLTK `PortStemmer` module.

```
sentence = "We were overwhelmed by the generosity of friends
and neighbours"
```

```
from nltk.stem import PorterStemmer
words = word_tokenize(sentence)
ps = PorterStemmer()
[ps.stem(word) for word in words]
['we',
'were',
'overwhelm',
'by',
'the',
'generos',
'of',
'friend',
'and',
'neighbour']
```

Frequency table

Extractive text summarisation requires a frequency table of word counts in a Python dictionary data form. This exercise requires both word tokenization and stemming to count each

word. The function `create_frequency_table(text_body)` will create a dictionary of words with their count as the key field. This function filters out the stop words and punctuations from the text body to make the frequency table more concise and meaningful.

```
[ps.stem(word) for word in words if word.lower() not in
['we', 'were', 'by', 'the', 'of', 'and']]
```

```
Out: ['overwhelm', 'generos', 'friend', 'neighbour']
```

```
def createFrequencyTable(textBody) -> dict:

    #Collect all stop words from NLTK
    stopWords = set(stopwords.words("english"))
    #Tokenization
    words = word_tokenize(textBody)
    #Filterout all stop words
    words = [w for w in words if w.lower() not in stopWords]
    #Filter out all punctuations
    words = [w for w in words if w.lower().isalpha()]
    #Call stemmer object
    ps = PorterStemmer()

    freqTable = dict()
    for word in words:
        word = ps.stem(word)
        if word in freqTable:
            freqTable[word] += 1
        else:
            freqTable[word] = 1

    return freqTable
```

Stop words

One of NLP’s core aspects is handling ‘stop words’, which due to their frequent occurrence in the text often don’t offer substantial insights on their own.

Stop words like ‘the’, ‘and’, and ‘I’, although common, don’t usually provide meaningful information about a document’s specific topic. Removing these words from a corpus allows us to identify unique and relevant terms more easily.

It’s important to note that there is no universally accepted list of stop words in NLP. However, the Natural Language Toolkit offers a powerful list of ‘stop words’ for researchers and practitioners to utilise.

```
from nltk.corpus import stopwords

stopWords = set (stopwords.words("english"))
```